

Jelcos AI — Executive Meeting Brief

Prepared 2026-02 | For 1:1 with Emergent executive | Confidential

Jelcos AI is a production-grade Expo + FastAPI + MongoDB application self-hosted on AWS EC2 with Cloudflare and MongoDB Atlas. 32 modules, 89 features, weekly regression suite, live admin panel with self-serve test runner, plus a tier-matrix subscription model. This brief consolidates the credit-burn, time-waste, and tech-limitation issues experienced on the Emergent platform, the available programs to address them, and the proposed win-win path forward.

Section A1 — Credit / Money Wastage (paid for fixes of agent-introduced bugs)

#	Issue	Root cause (honest)	Fault	Credit cost
1	Phantom pin requirements.txt (contourpy 1.3.3, jq 1.11.0, tiktoken 0.12.0, google-auth dev0). EC2 Docker build failed 2+ times.	Prior agent ran <i>pip freeze</i> and committed transient / aarch64-only pins. Bloat from 165 lines (dev tools mixed with runtime).	Agent	High
2	Missing <i>Platform</i> import in (tabs)/index.tsx — used Platform.OS without importing it.	Earlier session added check without import. Silent in dev (closure not evaluated), crashed in prod.	Agent	Medium
3	React #418 hydration mismatch hid 80% of homepage modules in prod (only ~5 visible out of 20+).	No agent ever added the standard <i>useEffect(() => setIsClient(true))</i> guard despite Expo Router web SSR being well known.	Agent	High
4	Weekly regression scheduler fired 4× (uvicorn --workers 4).	Original deploy ignored multi-worker reality. Standard <i>fcntl.flock</i> gating was missing.	Agent	Low-Med
5	Stale <i>user_id_1</i> Mongo index crashed seeder on first prod deploy.	Earlier session changed schema without dropping obsolete unique index.	Agent	Medium
6	AdminShell double ScrollView collapsed content to 0 height.	Polish session nested ScrollViews incorrectly.	Agent	Medium
7	Solution Matrix nested orgtypes — backend not built when UI shipped.	Prior session declared epic done while only UI was complete.	Agent	High
8	Voice Browsing logged as finished epic; only PRR scope working.	Agent over-promised scope in carry-forward note.	Agent	Medium
9	Frontend testing agent only completed 1 of 9 journeys.	Testing-agent harness rejects multi-step scripts >80 lines. Limitation surfaced AFTER call consumed credits.	Platform	Medium
10	LLM budget cap hit 9+ times across forks.	Universal Emergent LLM key has hard cap; not transparent before 503.	Platform	Medium

8 of 10 issues were agent-caused — not platform. The credit-refund / waiver ask in Section C is grounded in this.

Section A2 — Time Wasted (repeating same instructions across forks)

#	Repeated friction	Why it keeps happening	Fix that would save time
T1	Said "pull from emergent-v3, not main" multiple times	Forks don't preserve branch convention in handoff summary. Not surfaced in env or metadata.	Pin /app/memory/deployment.md with branch + EC2 path + build cmds
T2	Cited wrong GitHub-push button UI location	Emergent UI moved; agent training docs lag.	Maintain single source-of-truth UI map in support agent
T3	Re-explained "prod = jelcos.ai + api.jelcos.ai + Atlas" every fork	Handoff mentions stack but URL/host bindings aren't first-class.	Pin /app/memory/infrastructure.md
T4	Re-uploaded EC2 error screenshots multiple times	Forks drop image context; only text handoff carries.	Convert screenshots to text notes before fork
T5	Verify + finish + memo took 2 sessions instead of 1	Forks happen on token exhaustion; agents over-promise.	Bigger token budget per fork OR explicit user fork checkpoints
T6	Frontend testing agent needed 5 chunks instead of 1	Agent harness limit discovered only after credit spend.	Auto-chunking in testing agent
T7	Asked support agent for own UI location	Agent didn't know its platform UI.	Continuous internal training docs for UI knowledge

Section A3 — Real Tech Limitations of Emergent Platform (today)

#	Limitation	Impact on you	Strategic ask
L1	LLM budget cap on universal key	AI Chat, CLD, AI Research, Public Pulse Phase 3 silently 503.	Higher tier cap, BYO-key option, transparent burn meter
L2	Fork context loss	Long threads truncated; images dropped; prior decisions blurred.	Persistent project memory layer, longer context tier
L3	Testing agent script-size cap	Can't run full 9-journey smoke suite in one call.	Stable harness with chunking built-in
L4	No CI/CD hook into Save to GitHub	Push success known only by waiting + refreshing GitHub.	Webhook-triggered build status display
L5	No git status / branch visible in agent IDE	Wrong-branch mistakes (T1).	Branch-aware UI
L6	No pre-commit pip-install smoke test	Phantom pins reach prod before being caught.	Optional CI pre-commit test of requirements.txt
L7	Subscription/billing reflection delay	Just-paid users see Upgrade banner — looks like fraud.	Real-time Stripe webhook + force-refresh button
L8	No credit-forecast / estimate before run	Surprise burns.	Estimate-before-run option for agent tasks
L9	No multi-LLM key isolation	Universal cap hits all 3 providers simultaneously.	Per-provider key isolation

Section B — Emergent Programs Available Today (honest assessment from support)

Program	Exists today?	How to enroll / next step
Free / Trial plan	YES	Sign up
Pro plan (\$20/mo) — you are on this	YES	In-app upgrade
Team plan	YES	Contact support@emergent.sh
Top-up credits (one-off packs)	YES	In-app purchase
Dedicated Customer Success / Account Manager	NOT PUBLIC	Ask the executive directly
Enterprise / Founder / Build Partner program	NOT PUBLIC	Ask the executive directly
Credit refund policy for agent-introduced bugs	NO POLICY	Bring evidence; request as one-off goodwill
Higher LLM budget tier	NOT PUBLIC	Ask the executive directly
BYO LLM key (OpenAI/Anthropic/Gemini)	NOT SUPPORTED	Ask as roadmap item
Early-access / roadmap program	NOT PUBLIC	Ask the executive directly
Startup / accelerator credits	NOT PUBLIC	Ask the executive directly
Escalation email	YES	support@emergent.sh

Section C — What to Bring + What to Ask

1. Quantified evidence

- Total credits consumed to date (from billing dashboard)
- Credits in the last 4 forks, labelled by feature
- Estimated % spent on agent-introduced-bug fixes vs. forward features (target: pin a number, e.g. 35–45%)

2. The 3 hardest pain stories (with timestamps and screenshots)

- Production Docker build failed twice from phantom pins (A1.#1)
- Hydration bug hiding 80% of modules from real users (A1.#3)
- Subscription paid but dashboard still asks for payment (today's incident)

3. Your specific asks — be direct

- Credit waiver / refund for agent-introduced-bug spend
- Higher LLM budget tier OR BYO-key option
- Persistent project memory so forks stop losing context
- Direct line / Slack channel to one Emergent engineer
- Early access to fork-context improvements (you'll be a design partner)
- Roadmap visibility on testing-agent harness, billing-reflection latency

4. Your win-win offer (what you bring to the table)

- A live production app on AWS + Cloudflare + Atlas using Emergent end-to-end
- High-value reference customer (32 modules, 89 features, weekly regression)
- Willingness to be a design partner, write a case study, refer founders

Section D — Strategic Tactics to Ship Cost-Effectively on Emergent (going forward)

Tactic	Saves...
Pin /app/memory/infrastructure.md + /app/memory/deployment.md with branch, EC2 path, build cmds, prod URLs	Stops T1–T3 repetition forever
Before every fork, ask agent for a one-page carry-forward covering all in-flight epics + first test in next session	Recovers T5-type loss
Convert screenshots → text notes in /app/memory/issues.md immediately after each debug round	Survives image-drop on fork (T4)
Always ask for a credit estimate before approving a big task (esp. testing agent calls)	Caps surprise burn
For multi-step testing, ask agent to chunk into 5+ smaller calls upfront	Avoids L3 waste
For deployments, run a local <i>pip install --dry-run --platform manylinux2014_x86_64</i> before pushing to EC2	Catches L6 phantom pins
Dedicated emergent-v3 rebase workflow; never touch main directly	Avoids T1
Run regression suite weekly via the in-app dashboard, not via agent calls	Reuses what's built — zero extra credits
Front-load architecture / DB schema decisions at session start (low-credit phase) before deep feature work	Reduces rework

Honest Disclaimers (so you don't walk in over-promising)

1. Enterprise, Founder, Account Manager, and credit-refund programs are **not publicly documented**. Ask the executive directly; do not assume yes.
2. 8 of the 10 Section-A1 issues were agent-caused, not platform-caused. This is a fair goodwill-credit ask, NOT an entitlement under any documented policy.
3. Tech limitations in Section A3 are real platform ceilings — treat as roadmap-bargaining items, not fix-it-today items.

Recommended Order of Topics in the Meeting (30-minute slot)

- 0-2 min:** Your opening pitch (separate file: opening_pitch.md)
- 2-7 min:** Show Section A1 — quantified credit waste; ask for goodwill credit waiver
- 7-12 min:** Show Section A3 (limitations) — ask for higher LLM tier / BYO key / persistent memory
- 12-18 min:** Show Section A2 (time waste) — ask for Account Manager + roadmap access
- 18-23 min:** Reference your win-win offer — design partner, case study, referrals
- 23-28 min:** Concrete commitments — "by when can I expect X?" Pin dates if possible
- 28-30 min:** Wrap with thank-you + agreed next-step owner